



Веб-система управління дефектами програмного забезпечення з механізмом аналізу архітектурного впливу

Виконав: ст. гр. ПЗПІ-23-1 Булгаков М.О.
Керівник: доц. каф. ПІ Голян Н.В.

Мета роботи

Розробка комплексної веб-системи управління дефектами ПЗ із вбудованим механізмом аналізу архітектурного впливу для мінімізації ризиків та оптимізації регресійного тестування.

Актуальність теми:

- Масове залучення ШІ-агентів (Copilot, Cursor і т.д.) до написання коду. Вони вирішують точкові задачі, але часто не здатні передбачити каскадний вплив своїх змін на складну архітектуру всього застосунку.
- Традиційні баг-трекери розглядають дефекти ізольовано. Виникає потреба в інструментах, що автоматично вираховують Regression Scope.

Аналіз існуючих рішень: Jira

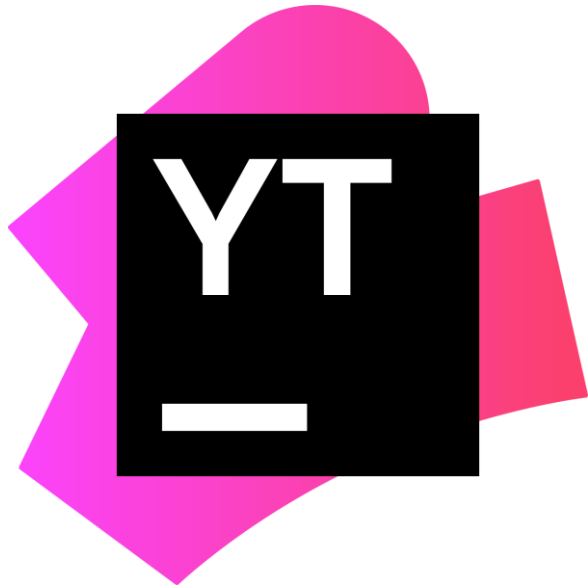


Тип: універсальна платформа для управління проєктами (Agile/Scrum).

Спрямованість: task-centric (орієнтація на задачі та бізнес-вимоги).

Прогалини: відсутність нативного аналізу архітектури коду; неможливість візуалізувати зв'язки між модулями та генерувати тест-кейси без купівлі сторонніх плагінів (наприклад, Zephyr).

Аналіз існуючих рішень: YouTrack



Тип: інтелектуальний баг-трекер та система управління задачами.

Спрямованість: глибока інтеграція з IDE та розширена система пошуку.

Прогалини: відсутність механізмів обчислення архітектурного впливу (Impact Analysis); суб'єктивна ручна оцінка критичності помилок без rule-based алгоритмів розрахунку ризику.

Аналіз існуючих рішень: Azure DevOps



Azure DevOps

Тип: комплексна корпоративна CI/CD платформа та баг-трекер.

Спрямованість: повний цикл SDLC (від контролю версій до тестових планів).

Прогалини: надмірна складність для малих/середніх команд; відсутність вбудованої інтеграції з LLM для швидкої AI-генерації регресійних тест-кейсів на основі описів дефектів.

Постановка задачі та опис системи

Проблеми:

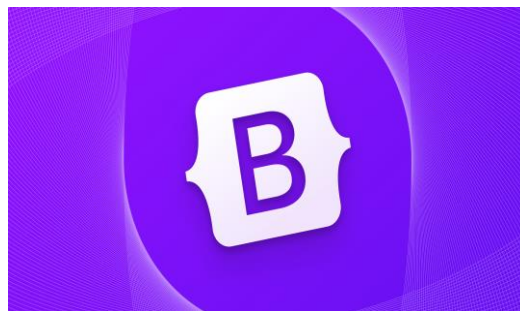
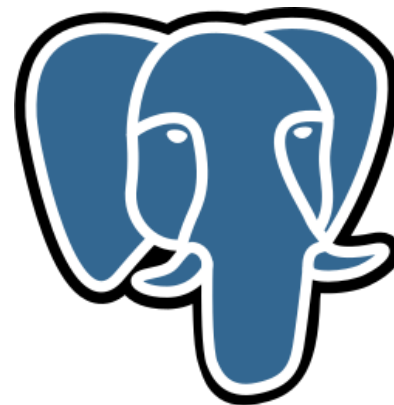
- традиційні системи розглядають баги ізольовано від загальної топології проєкту;
- відсутність інструментів автоматичного визначення області регресійного тестування;
- високі витрати часу QA-інженерів на написання рутинної тестової документації;
- суб'єктивна оцінка пріоритетів та ризиків помилок.

Очікувані результати:

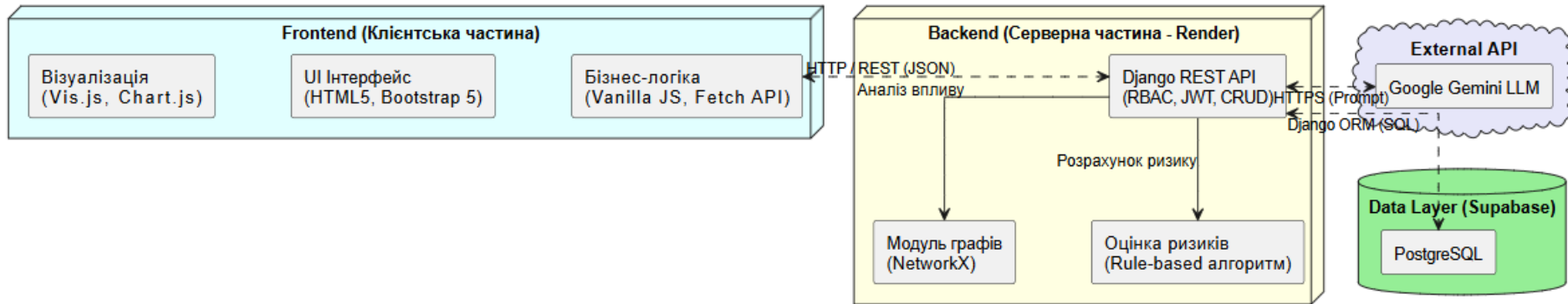
Розробка веб-застосунку, що забезпечить:

- математичний аналіз архітектури: побудова графа залежностей програмних модулів для виявлення каскадного впливу дефектів.
- Rule-based розрахунок ризиків: автоматичне обчислення Risk Score на основі критичності, історії перевідкриттів та кількості залежних модулів.
- інтеграцію з LLM: автоматичну генерацію структурованих тест-кейсів на базі тексту баг-репорту.
- управління процесом: рольову модель доступу (PM, QA, DEV), аналітичні дашборди та інтерактивну Kanban-дошку.

Вибір технологій розробки



Архітектура створеного програмного забезпечення



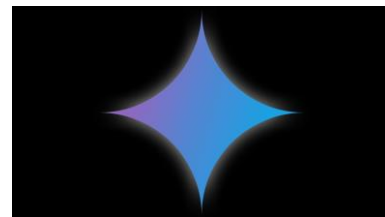
- **Модуль Accounts:** управління профілями, реєстрація, авторизація та Role-Based Access Control (RBAC).
- **Модуль Architecture:** Робота з проєктами, залежностями модулів (Impact Graph) та управління командою. Включає логіку Kanban-дошки.
- **Модуль Defects:** Баг-трекінг, історія життєвого циклу помилок, агрегація аналітики та Rule-based розрахунок ризиків (Risk Score).
- **Модуль QA_AI:** Формування контекстних prompt-запитів, зв'язок із моделлю Google Gemini та парсинг згенерованих JSON тест-кейсів.

Вибрані мови та фреймворки

- **Backend:** Python 3.10+, Django, Django REST Framework (DRF). Вибрані за потужну екосистему для обробки даних та вбудовану безпеку (ORM, JWT);
- **Frontend:** HTML5, CSS3 (Bootstrap 5), JavaScript (ES6). Забезпечують високу швидкість завантаження без перевантаження клієнта реактивними фреймворками;
- **База даних:** PostgreSQL (Supabase) — реляційна СУБД для гарантії цілісності транзакцій.

Спеціалізовані бібліотеки

- **NetworkX (Python):** Математичне моделювання спрямованих графів та пошук областей регресії;
- **Chart.js / Vis.js (JS):** Інтерактивна візуалізація дашбордів та архітектурних мереж;
- **Google Generative AI SDK:** Зв'язок із моделлю Gemini (LLM).



Опис процесу розробки ПЗ

- Проєктування: формування вимог (SRS), розробка UML-діаграм, проєктування реляційної БД;
- Реалізація Backend: створення REST API, інтеграція NetworkX та LLM, написання алгоритмів;
- Реалізація Frontend: верстка UI, підключення Fetch API, розробка Drag-and-Drop Kanban-дошки;
- Деплой: налаштування WSGI (Gunicorn), статичних файлів (WhiteNoise) та розгортання на хмарній платформі Render.

Дизайн системи

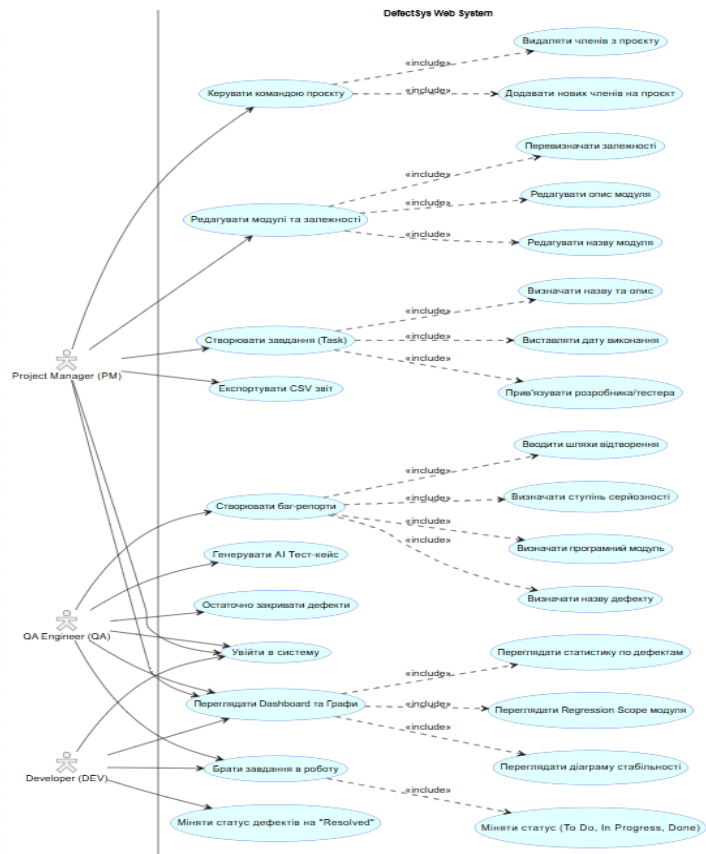
Методи проєктування:

- об'єктно-орієнтоване проєктування - розбиття системи на незалежні додатки (описані на слайді 8);
- візуальне моделювання (UML) – побудова діаграм прецедентів, компонентів, розгортання.

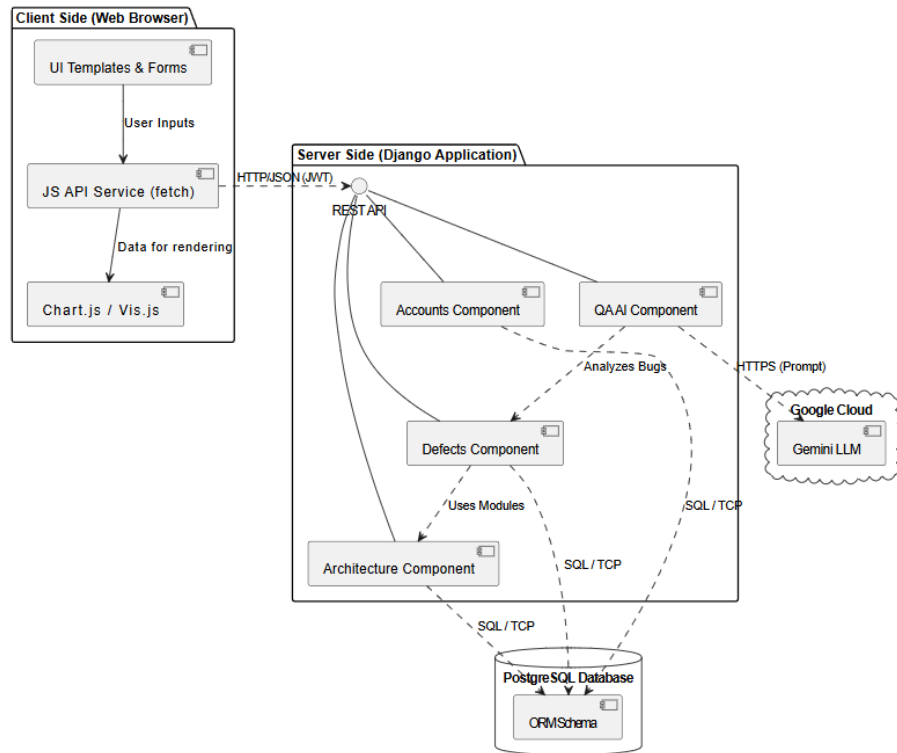
Технології та шаблони дизайну:

- патерн MVC/MVT (Model-View-Template) — базовий шаблон архітектури Django;
- stateless архітектура: передача стану авторизації через JWT-токени, без збереження сесій на сервері;
- UI/UX Дизайн: використання Responsive Design (адаптивність) на базі сітки Bootstrap.

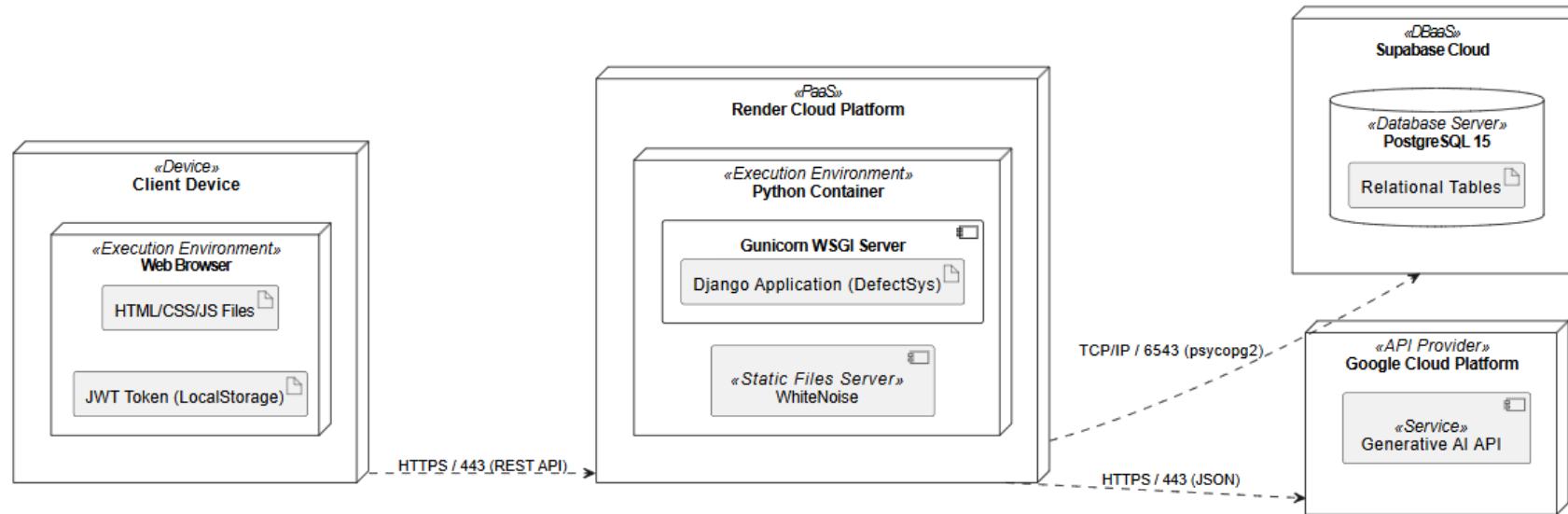
Діаграма прецедентів



Діаграма компонентів



Діаграма розгортання



Реалізація побудови графу залежностей та вираховування Regression Scope

```
5 def build_project_impact_graph(project_id: int) -> nx.DiGraph: 2 usages  ⌵ BulhakOv
6     G = nx.DiGraph()
7
8     modules = SoftwareModule.objects.filter(project_id=project_id)
9     for mod in modules:
10         G.add_node(mod.id, name=mod.name)
11
12     dependencies = ModuleDependency.objects.filter(source_module__project_id=project_id)
13
14     for dep in dependencies:
15         G.add_edge(dep.target_module_id, dep.source_module_id)
16     return G
17
18
19 def get_regression_scope(module_id: int) -> list: 2 usages  ⌵ BulhakOv
20     try:
21         module = SoftwareModule.objects.get(id=module_id)
22     except SoftwareModule.DoesNotExist:
23         return []
24
25     G = build_project_impact_graph(module.project_id)
26
27     if module_id not in G:
28         return []
29
30     impacted_module_ids = list(nx.descendants(G, module_id))
31     impacted_modules = SoftwareModule.objects.filter(id__in=impacted_module_ids)
32     result = [
33         {"id": m.id, "name": m.name}
34         for m in impacted_modules
35     ]
36
37     return result
```

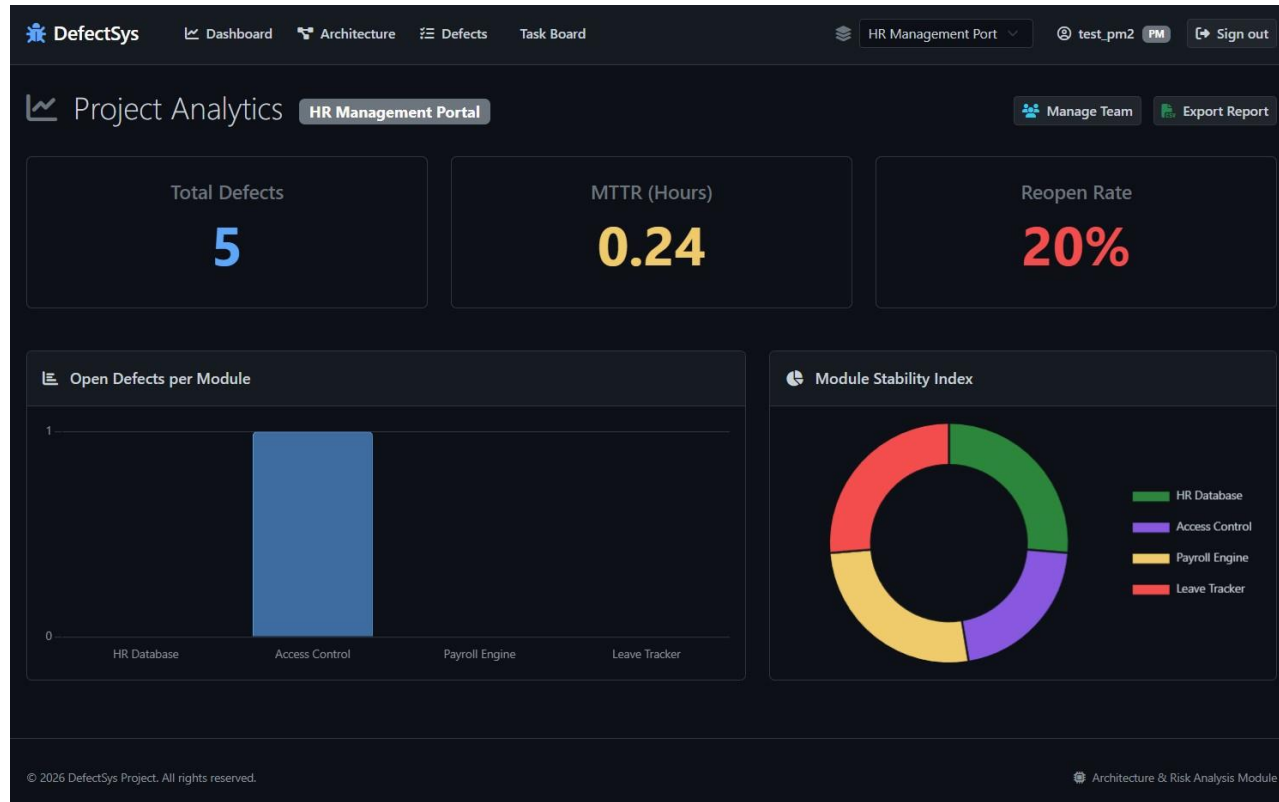

Реалізація підключення до Gemini API з подальшим промптом для формування тест-кейсів

```
8 def generate_and_save_test_case(defect_id: int, user) -> AITestCase: 2 usages  Δ BulhakOv
9     api_key = getattr(settings, 'GEMINI_API_KEY', None)
10    if not api_key:
11        print("CRITICAL ERROR: GEMINI_API_KEY is not set in settings.py or .env file!")
12        raise ValueError("API Key for AI is missing.")
13
14    genai.configure(api_key=api_key)
15
16    try:
17        defect = Defect.objects.select_related('module').get(id=defect_id)
18    except Defect.DoesNotExist:
19        raise ValueError("Defect not found.")
20
21    if hasattr(defect, 'ai_test_case'):
22        return defect.ai_test_case
23
24    prompt = f"""
25    You are a Senior QA Engineer. Generate a basic regression test case
26    based on this bug report.
27
28    Defect Title: {defect.title}
29    Module: {defect.module.name}
30    Steps to Reproduce: {defect.steps_to_reproduce}
31
32    Return the result STRICTLY as a valid JSON object without any Markdown formatting (no ```json).
33    Use exactly this structure:
34    {{
35        "test_case_title": "string",
36        "preconditions": "string",
37        "steps_to_test": ["step 1", "step 2"],
38        "expected_result": "string",
39        "regression_recommendations": "string"
40    }}
41    """
```

Реалізація Rule-based алгоритму оцінку критичності дефекту

```
4  def calculate_defect_risk(defect) -> float: 4 usages  BulhakOv
5      risk_score = 0.0
6
7      # Severity
8      severity_weights = {
9          'Low': 10.0,
10         'Medium': 30.0,
11         'High': 60.0,
12         'Critical': 85.0
13     }
14     risk_score += severity_weights.get(defect.severity, 10.0)
15
16     # Reopen Rate
17     risk_score += (defect.reopen_count * 10.0)
18
19     # Impact Analysis
20     dependent_modules_count = ModuleDependency.objects.filter(
21         target_module=defect.module
22     ).count()
23
24     risk_score += (dependent_modules_count * 5.0)
25
26     return min(risk_score, 100.0)
```

Інтерфейс користувача: сторінка аналітики



Інтерфейс користувача: сторінка архітектури проєкту

DefectSys

DashboardArchitectureDefectsTask Board

HR Management Porttest_pm2 PMSign out

System Modules

View Impact Graph

Software Modules

+ Add

Name	Stability	Action
Leave Tracker	100%	
HR Database	100%	
Access Control	80%	
Payroll Engine	100%	

Create Dependency

Module (Depends on)Target (Core Module)

Select...Select...✓

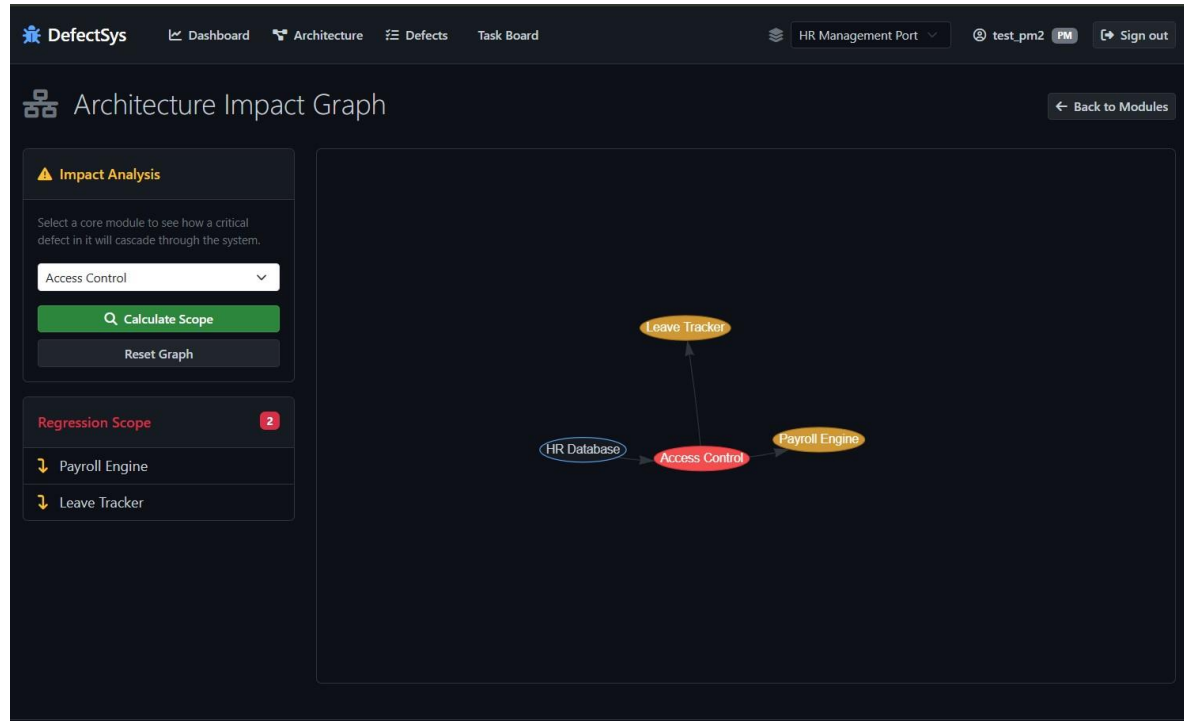
Existing Dependencies

Access Control	→	HR Database	×
Payroll Engine	→	Access Control	×
Leave Tracker	→	Access Control	×

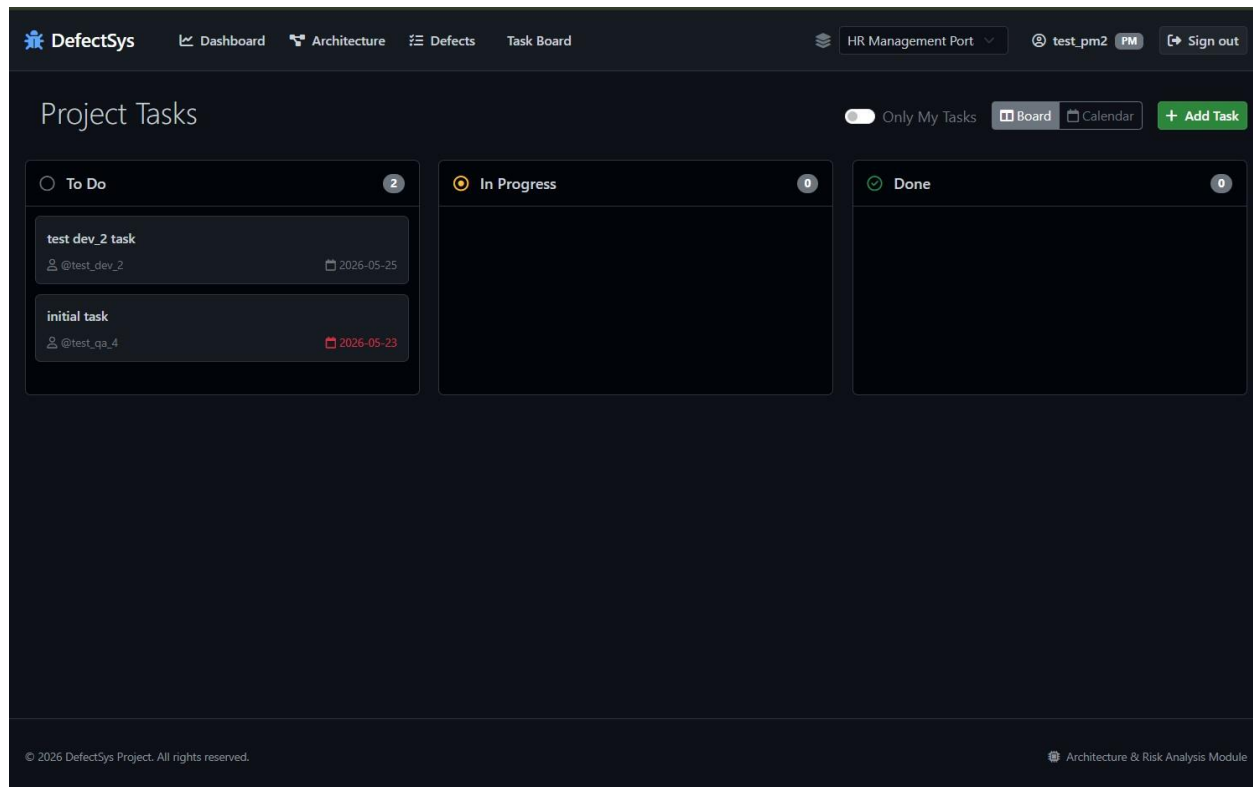
© 2026 DefectSys Project. All rights reserved.

Architecture & Risk Analysis Module

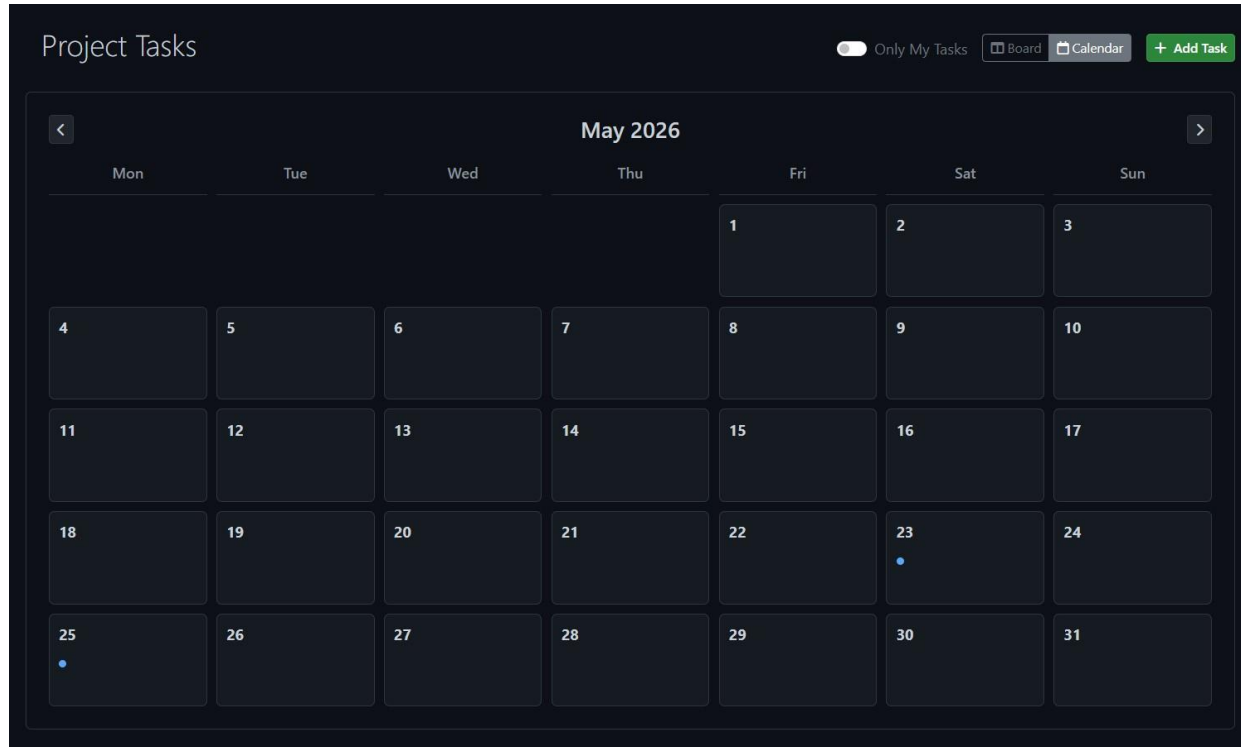
Інтерфейс користувача: сторінка із графом залежностей програмних модулів



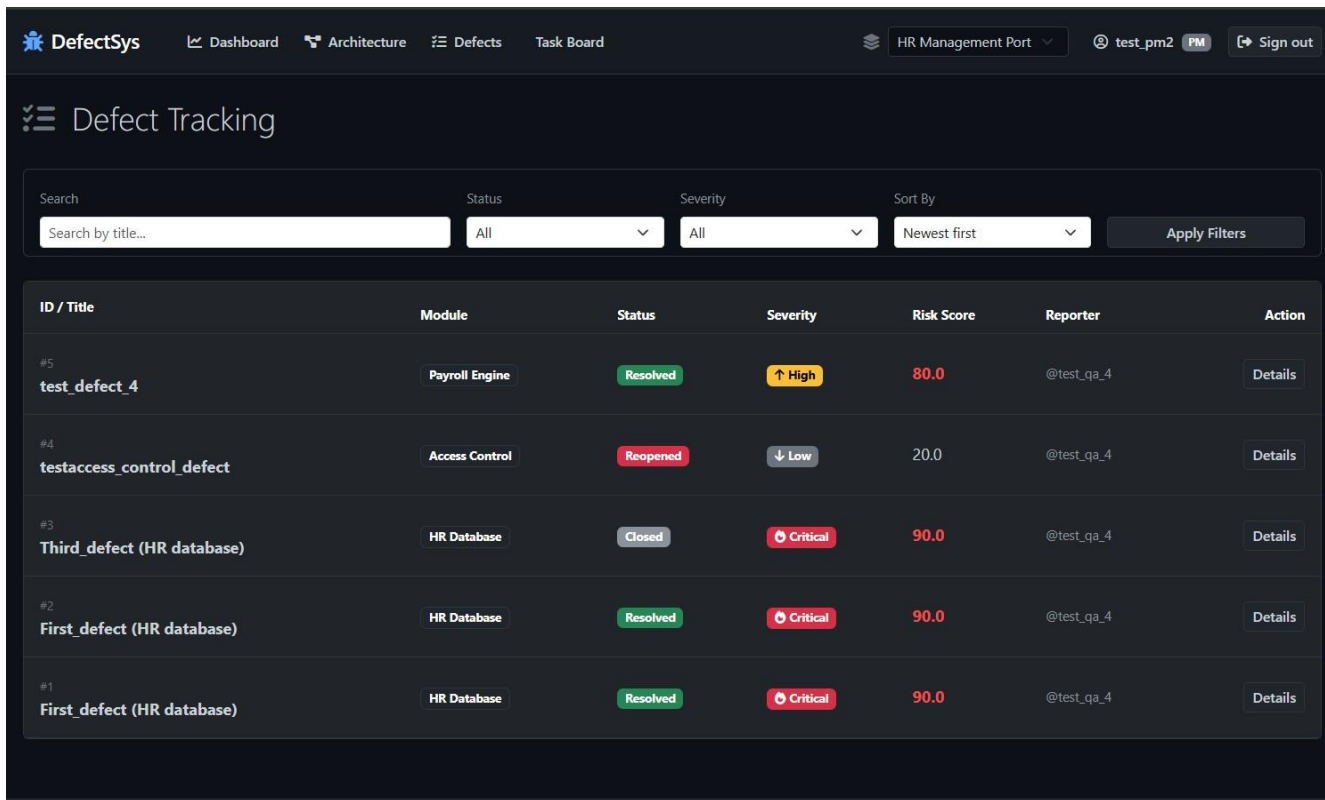
Інтерфейс користувача: сторінка Kanban-дошки



Інтерфейс користувача: сторінка з календарним відображенням завдань



Інтерфейс користувача: сторінка списку дефектів



The screenshot displays the DefectSys application interface. At the top, a navigation bar includes links for Dashboard, Architecture, Defects, and Task Board. The current page is titled "Defect Tracking". Below the title, there is a search bar and filters for Status (All), Severity (All), and Sort By (Newest first). An "Apply Filters" button is also present. The main content area is a table listing five defects, each with a unique ID, title, module, status, severity, risk score, reporter, and an action button.

ID / Title	Module	Status	Severity	Risk Score	Reporter	Action
#5 test_defect_4	Payroll Engine	Resolved	↑ High	80.0	@test_qa_4	Details
#4 testaccess_control_defect	Access Control	Reopened	↓ Low	20.0	@test_qa_4	Details
#3 Third_defect (HR database)	HR Database	Closed	🔥 Critical	90.0	@test_qa_4	Details
#2 First_defect (HR database)	HR Database	Resolved	🔥 Critical	90.0	@test_qa_4	Details
#1 First_defect (HR database)	HR Database	Resolved	🔥 Critical	90.0	@test_qa_4	Details

Інтерфейс користувача: сторінка із деталями дефекту

DefectSys

DashboardArchitectureDefectsTask Board

HR Management Port

test_pm2 PM

Sign out

[← Back to list](#)

#5 test_defect_4 Resolved

Details & Steps to Reproduce

Reported by @test_qa_4

TARGET MODULE

Payroll Engine

SEVERITY

↑ High

RISK SCORE

80.0

STEPS TO REPRODUCE

1. go to the payment page
2. enter the credit card info
3. api error

AI Test Case Generator

REG-PAY-001: Verify successful credit card payment processing after API error fix

PRECONDITIONS

User is logged in as an employee/administrator with necessary permissions to access payment functionality. A valid credit card with sufficient funds is available. System is configured for credit card payments and the API gateway is operational.

STEPS TO TEST

1. Navigate to the Payroll Engine payment page.
2. Enter valid credit card details (e.g., card number, expiry date, CVV, cardholder name) and a valid payment amount.

Update Status

Resolved

Update Defect

Lifecycle History

2 reopens

@test_qa_4

23.05.2026, 22:26:25

Reopened → Resolved

@test_qa_4

23.05.2026, 22:25:26

Resolved → Reopened

@test_qa_4

23.05.2026, 22:21:24

Reopened → Resolved

@test_qa_4

23.05.2026, 22:19:26

Resolved → Reopened

@test_qa_4

23.05.2026, 22:19:12

In Progress → Resolved

@test_qa_4

23.05.2026, 22:18:50

New → In Progress

Підсумки

Реалістичність та корисність:

- Висока практична цінність: Система повністю замінює традиційні баг-трекери, автоматизуючи рутину QA-інженерів завдяки AI-генерації тест-кейсів;
- Готовність до експлуатації: Проєкт реалізовано за сучасними стандартами (REST API, PostgreSQL, Stateless JWT) та розгорнуто на хмарній платформі Render.

Можливості використання:

- IT-компанії та стартапи: для управління життєвим циклом розробки ПЗ.
- Команди тестувальників: Як інтелектуальний асистент для виявлення "сліпих зон" у мікросервісних архітектурах та об'єктивної оцінки технічного боргу.

Подальший розвиток системи

- **Real-Time взаємодія:** впровадження протоколу WebSocket для синхронного оновлення Kanban-дошки у всіх користувачів;
- **Інтеграція CI/CD:** автоматична зміна статусів дефектів при злитті гілок коду з GitHub / GitLab;
- **Оптимізація UI графів:** впровадження механізмів кластеризації вузлів для коректного рендерингу архітектур із великою кількістю компонентів.